

Отчёт по лабораторной работе №5

Аппарат сетей Петри

Сахно Никита НФИбд-02-23

Abstract

В данном отчёте представлены результаты выполнения лабораторной работы по теме «Аппарат сетей Петри» на примере задачи «Обедающие философы».

Описаны построение классической сети Петри и сети с арбитром, проведение стохастического моделирования, обнаружение взаимной блокировки deadlock, визуализация результатов, создание анимации и преобразование программы в литературный стиль с последующей генерацией скрипта, Jupyter Notebook и документации.

Введение

Актуальность

Сети Петри представляют собой математический аппарат для моделирования дискретных систем, в которых необходимо учитывать параллельность, асинхронность и конкуренцию за общие ресурсы. Они позволяют описывать состояние системы с помощью позиций и фишек, а изменение состояния — с помощью срабатывания переходов.

В лабораторной работе рассматривается классическая задача «Обедающие философы», демонстрирующая проблему взаимной блокировки при конкурентном использовании общих ресурсов. На её основе исследуется возникновение deadlock и способ его предотвращения с помощью дополнительной позиции-арбитра.

Цель работы

Целью лабораторной работы является построение сети Петри для пяти философов, моделирование захвата и освобождения вилок, обнаружение состояния взаимной блокировки, проведение имитационного моделирования и модификация сети для предотвращения deadlock.

Задачи

Для достижения цели необходимо:

— Создать рабочий каталог для кода. — Установить необходимые пакеты. — Выполнить предложенный код. — Преобразовать код в литературный стиль. — Сгенерировать из литературного кода: — чистый код; — jupyter notebook; — документацию в формате Quarto. — Выполнить код из jupyter notebook. — Интегрировать документацию в формате Quarto в отчёт. — Добавить в код в литературном стиле вычисление для набора параметров. — Сгенерировать из литературного кода с параметрами: — чистый код; — jupyter notebook; — документацию в формате Quarto. — Выполнить код из jupyter

notebook с параметрами. — Интегрировать документацию с параметрами в формате Quarto в отчёт

Объект и предмет исследования

Объектом исследования является дискретная система, представленная сетью Петри и описывающая взаимодействие нескольких философов, конкурирующих за общие ресурсы.

Предметом исследования являются переходы между состояниями философов, использование вилок, возникновение взаимной блокировки и влияние введения арбитра на функционирование системы.

Методы исследования

В работе использовались:

- математическое моделирование сетями Петри;
- стохастическое имитационное моделирование по алгоритму Гиллеспи;
- детерминированное моделирование на основе системы уравнений;
- программная реализация на языке Julia;
- анализ конечной маркировки сети;
- графическая визуализация результатов;
- генерация литературного кода с использованием Literate.jl;
- автоматическая генерация Jupyter Notebook и документации.

Теоретическая часть

Сети Петри

Сеть Петри является математическим аппаратом для моделирования дискретных систем. Графически она представляется двудольным ориентированным графом, состоящим из позиций и переходов.

Позиции описывают состояния системы или наличие ресурсов. Внутри позиций располагаются фишки, количество которых определяет текущую маркировку сети. Переходы описывают события или действия, изменяющие состояние системы. Дуги связывают позиции с переходами и определяют условия их срабатывания.

Переход является разрешённым, если во всех его входных позициях имеется необходимое количество фишек. При срабатывании перехода фишки удаляются из входных позиций и добавляются в выходные позиции.

Формально сеть Петри задаётся четвёркой

$$N = (P, T, F, M_0),$$

где P — множество позиций, T — множество переходов, F — множество дуг, а M_0 — начальная маркировка.

К основным свойствам сетей Петри относятся ограниченность, активность, достижимость и сохраняемость.

Задача «Обедающие философы»

Задача «Обедающие философы» была сформулирована Эдсгером Дейкстрой в 1965 году. За круглым столом находятся N философов и N вилок. Каждый философ может находиться в состояниях «думает», «голоден» и «ест».

Для приёма пищи философу необходимы две вилки: левая и правая. При этом одна вилка не может одновременно использоваться двумя соседними философами.

В классической постановке возникает deadlock: каждый философ может получить одну вилку и затем ожидать вторую, уже занятую соседним философом. В результате все философы оказываются заблокированными.

Классическая сеть Петри

Для каждого философа используются позиции:

- $Think_i$ — философ размышляет;
- $Hungry_i$ — философ ожидает вилки;
- Eat_i — философ ест;
- $Fork_i$ — соответствующая вилка свободна.

Для каждого философа задаются переходы:

- $GetLeft_i$ — получение левой вилки;
- $GetRight_i$ — получение правой вилки;
- $PutForks_i$ — освобождение обеих вилок.

Начальная маркировка содержит по одной фишке в каждой позиции $Think_i$ и $Fork_i$.

Сеть с арбитром

Для предотвращения deadlock в модель вводится дополнительная позиция $Arbiter$. В неё помещается $N - 1$ фишек. Получение первой вилки становится возможным только при наличии фишки арбитра. После освобождения вилок фишка возвращается арбитру.

Таким образом, одновременно не допускается ситуация, в которой все философы удерживают по одной вилке и ожидают вторую.

Стохастическое моделирование

Для стохастического моделирования использован алгоритм Гиллеспи. На каждом шаге рассчитываются интенсивности разрешённых переходов, выбирается следующий переход и случайная величина времени до его срабатывания.

В результате формируется временная последовательность маркировок сети, которая сохраняется в таблицу DataFrame и далее экспортируется в CSV-файл.

Материалы и методы

В работе использовалась следующая программная среда:

- язык программирования Julia;
- Julia 1.12.5;
- пакет OrdinaryDiffEq;
- пакет Plots;
- пакет DataFrames;
- пакет CSV;
- пакет Random;
- пакет Literate;
- Jupyter Notebook;
- Quarto.

Рабочий каталог проекта:

```
C:\Users\nikit\Desktop\immod\lab05
```

Структура проекта содержит каталоги:

```
data/  
docs/  
literate/  
notebooks/  
plots/  
report/  
scripts/  
src/
```

Основной модуль модели расположен в:

```
src/DiningPhilosophers.jl
```

Основной сценарий моделирования:

```
scripts/dining_philosophers.jl
```

Скрипт создания анимации:

```
scripts/dining_philosophers_animation.jl
```

Скрипт формирования итогового сравнительного графика:

```
scripts/dining_philosophers_report.jl
```

Литературный вариант программы:

```
literate/dining_philosophers.jl
```

Для базового эксперимента использовались параметры

```
N = 5  
tmax = 50.0
```

Для анимации использовались параметры

```
N = 3  
tmax = 30.0
```

Выполнение лабораторной работы

Создание и запуск основной модели

Была создана модель задачи «Обедающие философы» на языке Julia. В программе реализованы две сети: классическая сеть без арбитра и сеть с арбитром.

Фрагмент построения классической сети:

```
function build_classical_network(N::Int)  
    n_places = 4N  
    n_transitions = 3N  
    net = PetriNet(n_places, n_transitions)  
  
    for i = 1:N  
        net.place_names[i] = Symbol("Think_$i")  
        net.place_names[N+i] = Symbol("Hungry_$i")  
        net.place_names[2N+i] = Symbol("Eat_$i")  
        net.place_names[3N+i] = Symbol("Fork_$i")  
    end  
end
```

Для каждого философа создаются позиции состояний и переходы, связанные с получением и освобождением вилок.

Построение сети с арбитром

Для модифицированной сети добавляется дополнительная позиция:

```
function build_arbiter_network(N::Int)  
    n_places = 4N + 1  
    n_transitions = 3N  
    net = PetriNet(n_places, n_transitions)  
  
    ...  
  
    arbiter_idx = 4N + 1  
    net.place_names[arbiter_idx] = :Arbiter  
end
```

Начальная маркировка арбитра задаётся количеством $N - 1$ фишек:

```
u0[arbiter_idx] = N - 1
```

Выполнение стохастического моделирования

Для обеих сетей было выполнено стохастическое моделирование продолжительностью 50.0 единиц времени.

Фрагмент вызова:

```
df_classic = simulate_stochastic(net_classic, tmax)
df_arbiter = simulate_stochastic(net_arbiter, tmax)
```

Полученные данные сохранялись в CSV-файлы.

Проверка deadlock

Для определения взаимной блокировки использовалась функция:

```
deadlock_classic = detect_deadlock(
    df_classic,
    net_classic
)

deadlock_arbiter = detect_deadlock(
    df_arbiter,
    net_arbiter
)
```

Важным моментом является передача в detect_deadlock таблицы результатов моделирования и соответствующей сети.

Получение результатов

При выполнении программы получен следующий результат:

```
Классическая сеть: deadlock = true
Сеть с арбитром: deadlock = false

=== Результаты моделирования ===
Количество философов: 5
Время моделирования: 50.0
Классическая сеть: deadlock = true
Сеть с арбитром: deadlock = false
```

График классической сети

На рис. 2 представлен график эволюции маркировки классической сети.

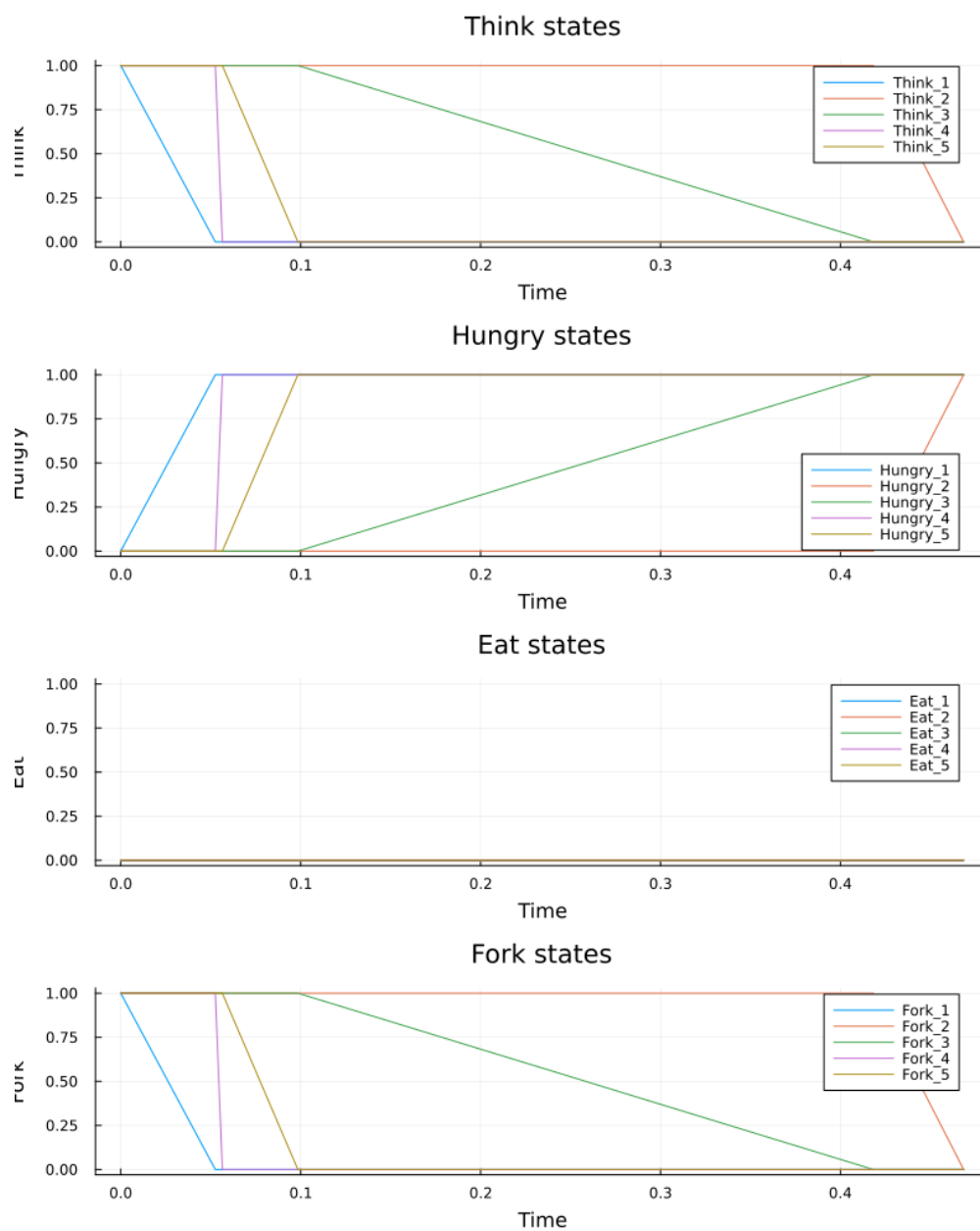


Figure 2: Эволюция маркировки классической сети Петри

График сети с арбитром

На рис. 3 представлен график эволюции маркировки сети с арбитром.

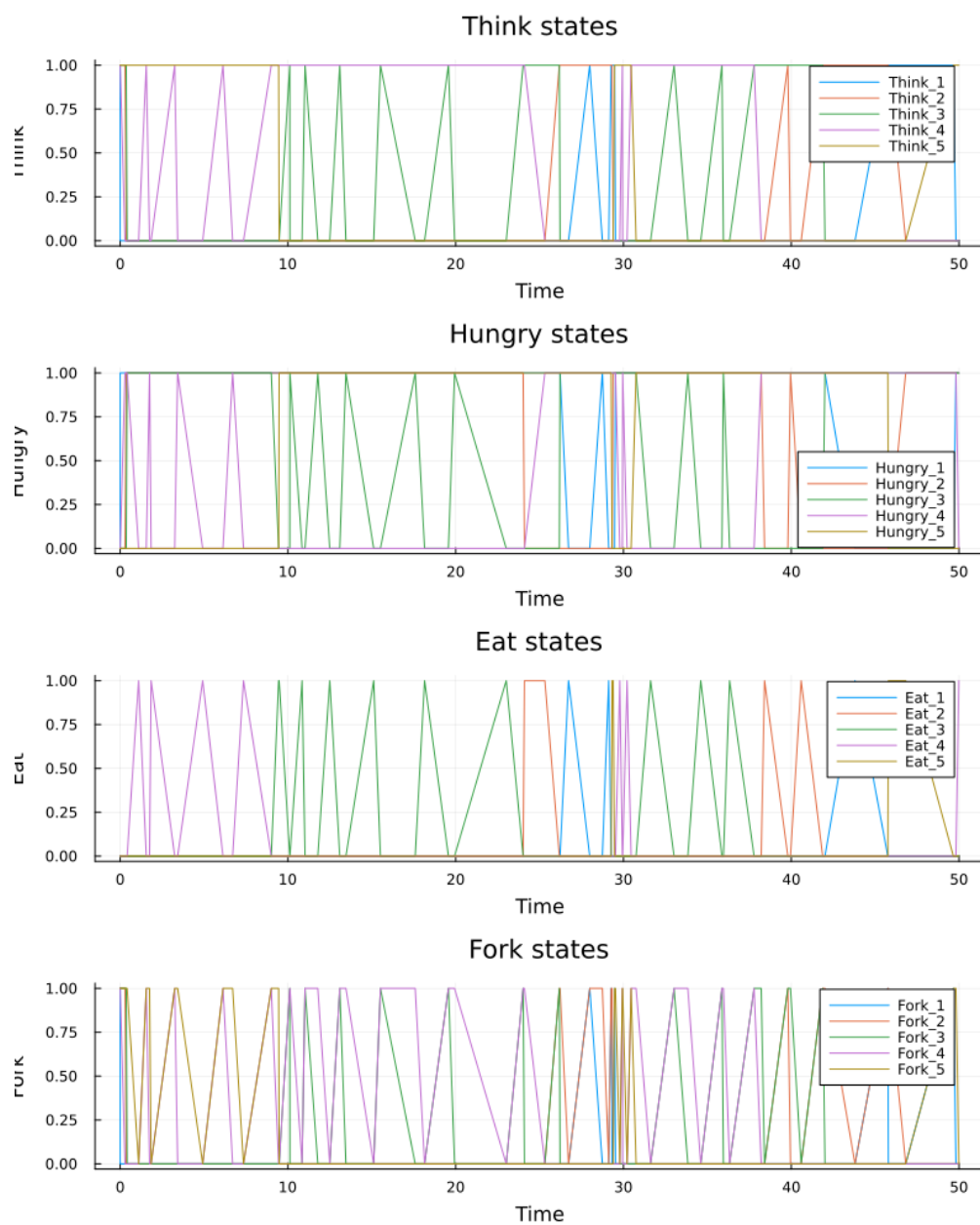


Figure 3: Эволюция маркировки сети Петри с арбитром

Итоговое сравнение

Для непосредственного сравнения двух вариантов модели был сформирован итоговый график.

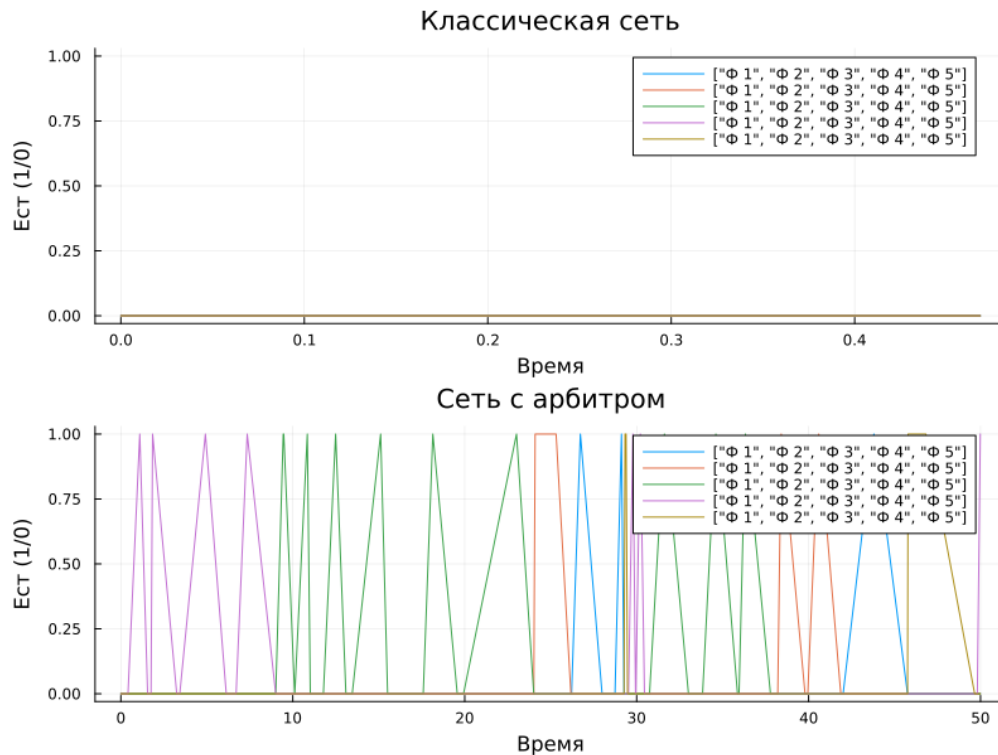


Figure 4: Сравнение классической сети и сети с арбитром

Литературный стиль

Исходный код был преобразован в литературный стиль. Для этого был создан файл:

```
literate/dining_philosophers.jl
```

В литературном коде выполняется подключение основного модуля:

```
include(joinpath(@__DIR__, "..", "src", "DiningPhilosophers.jl"))
using .DiningPhilosophers
```

После этого программа выполняет моделирование двух вариантов сети и сохраняет результаты.

Генерация чистого кода

Для генерации чистого Julia-кода использовалась команда:

```
Literate.script(
    "literate/dining_philosophers.jl",
    "scripts/generated"
)
```

В результате был создан файл:

```
scripts/generated/dining_philosophers.jl
```

Генерация Jupyter Notebook

Для генерации Notebook использовалась команда:

```
Literate.notebook(  
    "literate/dining_philosophers.jl",  
    "notebooks"  
)
```

В результате был создан:

```
notebooks/dining_philosophers.ipynb
```

При генерации Notebook код был автоматически выполнен Literate.

Генерация документации

Для создания документации использовалась команда:

```
Literate.markdown(  
    "literate/dining_philosophers.jl",  
    "docs"  
)
```

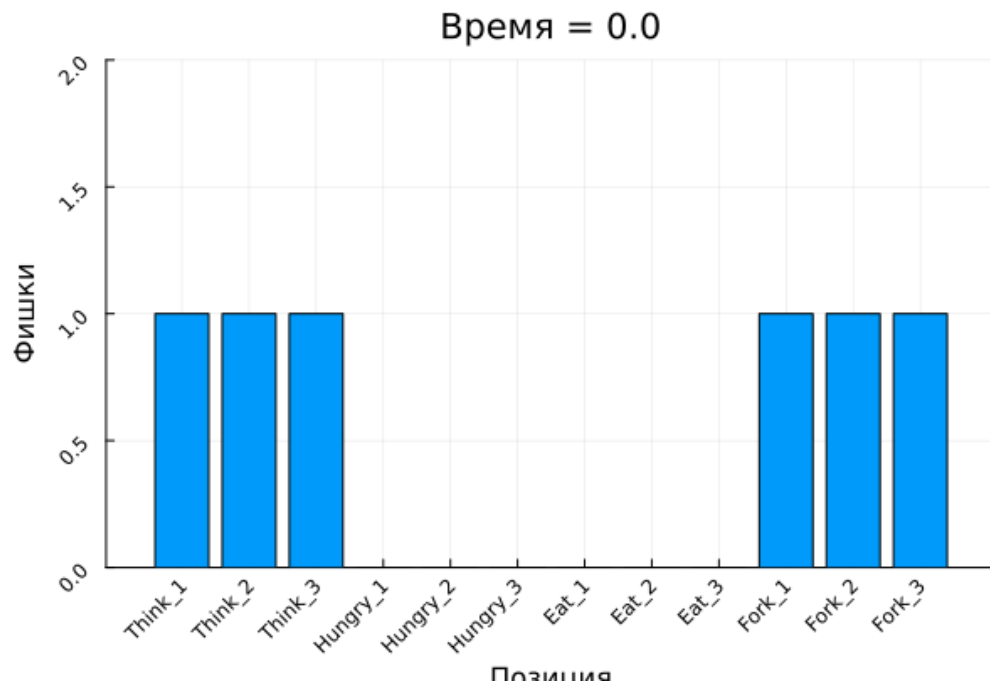
В результате был получен файл:

```
docs/dining_philosophers.md
```

Данный Markdown-файл предназначен для дальнейшего использования в Quarto.

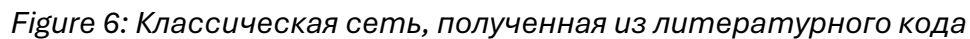
Дополнительная визуализация

Для демонстрации динамики сети была создана анимация процесса.



Анимация построена для трёх философов и демонстрирует изменение маркировки сети во времени.

В ходе выполнения литературного варианта были сформированы дополнительные графики:



Сеть Петри с арбитром

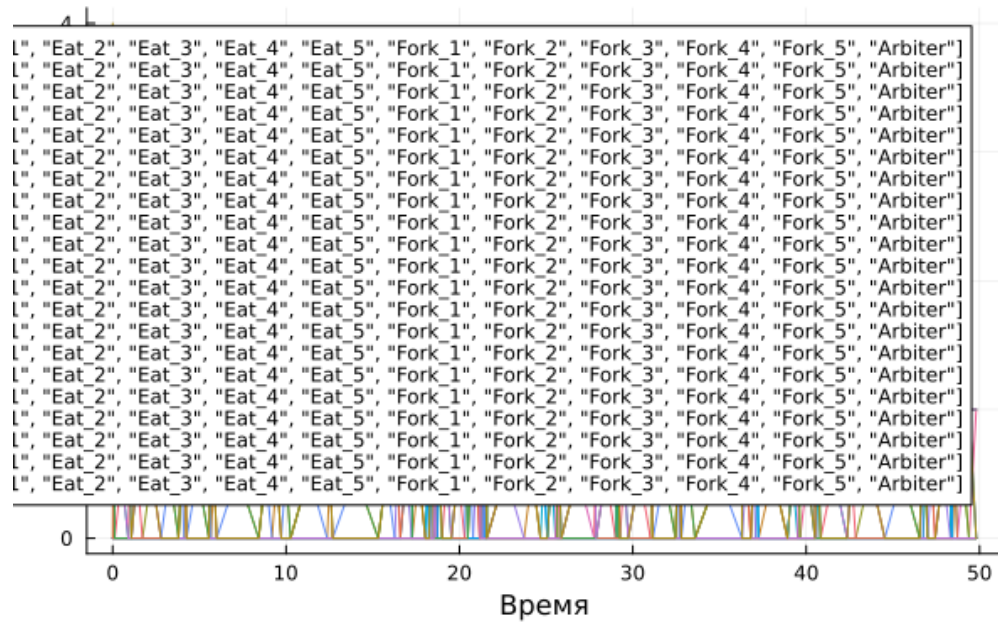


Figure 7: Сеть с арбитром, полученная из литературного кода

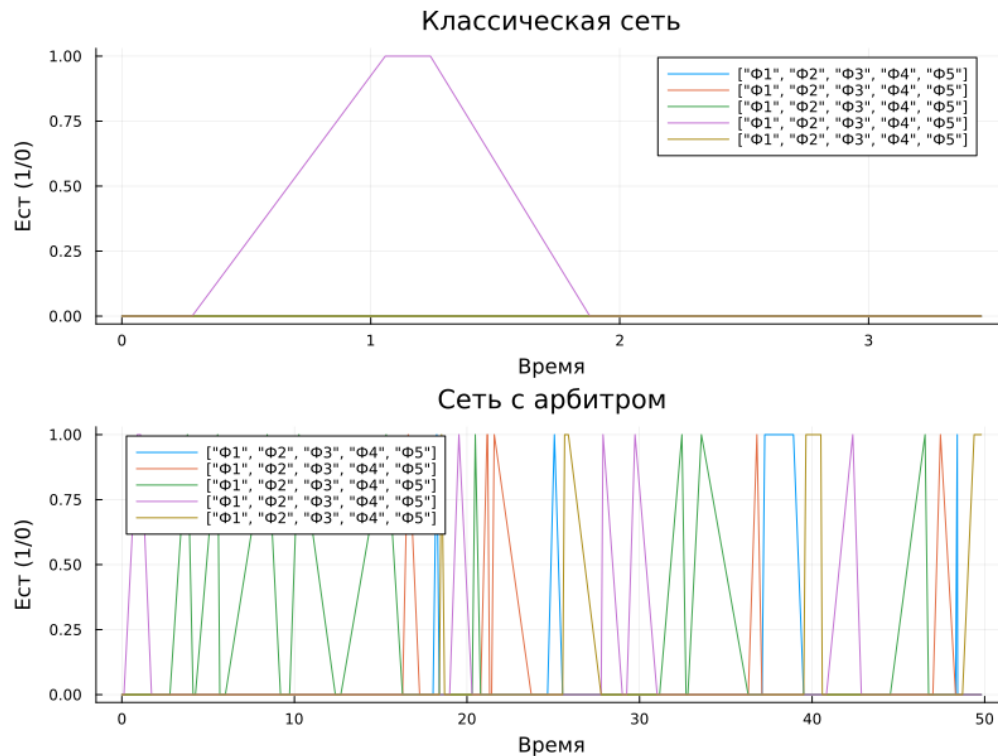


Figure 8: Сравнение результатов литературного варианта

Параметризованный вариант

В соответствии с заданием был подготовлен параметризованный литературный код, предусматривающий проведение вычислений для набора значений количества философов:

```
philosophers_set = [3, 4, 5, 6, 7]
tmax = 50.0
```

Для каждого значения N предусмотрено построение классической сети и сети с арбитром, выполнение стохастического моделирования и проверка deadlock.

Фрагмент цикла параметризованного расчёта:

```
for nphilosophers in philosophers_set

    net_classic = build_classical_network(nphilosophers)

    Random.seed!(123)
    df_classic = simulate_stochastic(net_classic, tmax)

    deadlock_classic = detect_deadlock(
        df_classic,
        net_classic
    )

    net_arbiter = build_arbiter_network(nphilosophers)

    Random.seed!(123)
    df_arbiter = simulate_stochastic(net_arbiter, tmax)

    deadlock_arbiter = detect_deadlock(
        df_arbiter,
        net_arbiter
    )
end
```

На момент подготовки отчёта генерация параметризованного Jupyter Notebook завершилась синтаксической ошибкой Literate при разборе блока for. Поэтому результаты параметризованного расчёта в отчёт не включаются как выполненные экспериментальные данные.

Результаты

В результате выполнения базового эксперимента для пяти философов и времени моделирования 50.0 получены следующие значения:

Параметр	Значение
Количество философов	5

Параметр	Значение
Время моделирования	50.0
Deadlock классической сети	true
Deadlock сети с арбитром	false

Для классической сети были сформированы данные:

```
data/dining_classic.csv
```

Для сети с арбитром:

```
data/dining_arbiter.csv
```

Были сформированы графические результаты:

```
plots/classic_simulation.png
plots/arbiter_simulation.png
plots/final_report.png
plots/classic_literate.png
plots/arbiter_literate.png
plots/literate_comparison.png
plots/philosophers_simulation.gif
```

Также были сформированы:

```
scripts/generated/dining_philosophers.jl
notebooks/dining_philosophers.ipynb
docs/dining_philosophers.md
```

Обсуждение результатов

Полученные результаты соответствуют ожидаемому поведению рассматриваемых сетей Петри.

Для классической сети функция обнаружения блокировки вернула true. Это означает, что в конечной маркировке не осталось разрешённых переходов и система оказалась в состоянии deadlock.

Для сети с арбитром функция обнаружения блокировки вернула false. Дополнительная позиция Arbiter, содержащая $N - 1$ фишек, ограничивает число философов, которые могут одновременно захватывать ресурсы, и предотвращает рассматриваемую взаимную блокировку.

На графиках классической сети наблюдается переход философов в состояния ожидания и прекращение дальнейших изменений маркировки после возникновения тупиковой ситуации. В модифицированной сети изменение маркировки продолжается, что соответствует отсутствию обнаруженного deadlock.

Таким образом, введение арбитра изменяет структуру сети и устраняет рассмотренный сценарий взаимной блокировки.

Отдельно была выполнена автоматизация преобразования литературного кода. Один исходный файл позволил получить чистый Julia-скрипт, Jupyter Notebook и Markdown-документацию, пригодную для дальнейшей обработки средствами Quarto.

Заключение

- Построена модель задачи «Обедающие философы» с использованием сетей Петри.
- Реализована классическая сеть для пяти философов.
- Реализована модифицированная сеть с арбитром.
- Выполнено стохастическое моделирование с временем моделирования 50.0.
- Для классической сети обнаружен deadlock: true.
- Для сети с арбитром deadlock не обнаружен: false.
- Получены CSV-файлы с траекториями маркировок.
- Построены графики изменения состояний сети.
- Создана GIF-анимация динамики сети.
- Исходная программа преобразована в литературный стиль.
- Сгенерированы чистый Julia-код, Jupyter Notebook и Markdown-документация.
- Подготовлен параметризованный вариант программы для расчётов при различных значениях количества философов; его автоматическая генерация Notebook требует исправления синтаксической ошибки Literate.

Список литературы

Листинг программы

Приложение А. Основной код модели

```
module DiningPhilosophers

using OrdinaryDiffEq
using Plots
using DataFrames
using Random, LinearAlgebra

export build_classical_network, build_arbiter_network
export simulate_ode, simulate_stochastic
export detect_deadlock, plot_marking_evolution

struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}
    place_names::Vector{Symbol}
```

```
transition_names::Vector{Symbol}  
end
```

Приложение Б. Запуск моделирования

```
N = 5  
tmax = 50.0  
  
net_classic = build_classical_network(N)  
  
Random.seed!(123)  
df_classic = simulate_stochastic(net_classic, tmax)  
  
deadlock_classic = detect_deadlock(  
    df_classic,  
    net_classic  
)  
  
net_arbiter = build_arbiter_network(N)  
  
Random.seed!(123)  
df_arbiter = simulate_stochastic(net_arbiter, tmax)  
  
deadlock_arbiter = detect_deadlock(  
    df_arbiter,  
    net_arbiter  
)  
  
println("Классическая сеть: deadlock = ", deadlock_classic)  
println("Сеть с арбитром: deadlock = ", deadlock_arbiter)
```

Приложение В. Генерация литературного кода

```
using Literate  
  
Literate.script(  
    "literate/dining_philosophers.jl",  
    "scripts/generated"  
)  
  
Literate.notebook(  
    "literate/dining_philosophers.jl",  
    "notebooks"  
)  
  
Literate.markdown(  
    "literate/dining_philosophers.jl",  
    "docs"  
)
```